Orientation for Python Developers

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Orientation for Python Developers
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 Welcome, Python Developers 1
1.1 What You Will Learn 1
1.2 Your Python Journey Starts Here 1

2 Introduction to InterSystems IRIS Development Using Python 3
2.1 What Is InterSystems IRIS? 3
2.2 Why InterSystems IRIS is Great for Python Developers 3
2.3 What You Can Build with InterSystems IRIS 4

3 Deciding How to Use InterSystems IRIS with Python 5
3.1 Python Options 5
3.1.1 Client-Server Options 6
3.1.2 Embedded Python 6
3.2 DB-API 7
3.2.1 When to Use the DB-API Driver 7
3.3 pyODBC 7
3.3.1 When to Use pyODBC 7
3.4 Native API 8
3.4.1 When to Use the Native API 8
3.5 Python Gateway 8
3.5.1 When to Use the Python Gateway 9
3.5.2 PEX Framework: A Key Use Case for the Python Gateway 9
3.6 Embedded Python 9
3.6.1 InterSystems IRIS Running Inside the Python Process 11
3.6.2 Python Running Inside the InterSystems IRIS Process 11
3.7 Choosing the Right Path for You 12
3.7.1 Python Support Capabilities 12
3.7.2 Python Uses Cheat Sheet 12

4 Use InterSystems IRIS as a Relational Database with DB-API 13
4.1 Prerequisites 13
4.2 DB-API: Direct SQL Access 13
4.3 Establishing a DB-API Connection 14
4.4 Executing a SQL Query 14
4.5 Parameters 15
4.5.1 Positional Parameters 15
4.5.2 Named Parameters 15

5 Access Relational Data as Objects with SQLAlchemy 17
5.1 Prerequisites 17
5.2 SQLAlchemy: SQL Toolkit and ORM 17
5.3 Establishing an Engine Connection 17
5.4 Transactions and Sessions 18
5.4.1 engine.begin() 18
5.4.2 engine.connect() 18
5.4.3 Session(engine) 19
5.5 SQLAlchemy Core: Fine-Grained SQL Control 19
5.5.1 Defining Tables with SQL Alchemy Core 19
5.6 SQLAlchemy ORM: Object-Oriented Data Access 20

5.6.1 Creating ORM Models 20
5.6.2 Saving Data 21
5.6.3 Retrieving Data 21
5.6.4 Deleting Data 22
5.6.5 Running the Script 22

6 Build an Interactive Application with Streamlit 25
6.1 Prerequisites 25
6.2 Building Interactive Apps with Streamlit and InterSystems IRIS 25
6.3 Building Your First Streamlit InterSystems IRIS App 26
6.4 What You Build (at a Glance) 26
6.5 Connecting to InterSystems IRIS 27
6.6 Configuring the Streamlit App Interface 28
6.7 Entering Queries 28
6.8 Executing SQL Queries 28
6.9 Displaying Query Results in a Table 29
6.10 Downloading Query Results as CSV Files 29
6.11 Visualizing Data 30
6.11.1 Retrieving and Validating Data 30
6.11.2 User Input—Column and Chart Type Selection 30
6.11.3 Generating Chart 31
6.11.4 Rendering Chart 31
6.12 Uploading CSV Data into InterSystems IRIS 31
6.12.1 Uploading a CSV 31
6.12.2 Previewing the Data 32
6.12.3 Entering Target Table Name 32
6.12.4 Inserting Data into InterSystems IRIS 32
6.13 Running Your Application 33
6.14 Complete Streamlit Code 33

7 Build a RESTful Application with Flask 37
7.1 Building RESTful Flask Applications with InterSystems IRIS 37
7.2 Prerequisites 37
7.3 What You Learn 37
7.4 What You Build 38
7.5 Creating Your First Flask App 39
7.6 Running the Flask Application 39
7.7 Using HTML Templates 39
7.8 Directory Structure 40
7.8.1 What Each Directory Component Is 40
7.9 Connecting to InterSystems IRIS 41
7.10 Transferring Data from InterSystems IRIS to Flask and Displaying it 41
7.10.1 tables.html 41
7.10.2 app.py 42
7.11 Complete Flask Code 44

8 Code Interactively with Jupyter Notebooks 47
8.1 Interactive Exploration with Jupyter Notebooks 47

9 Become an InterSystems IRIS Power User 49
9.1 Building Interoperability Productions with Python 49
9.1.1 Production EXtension Framework (PEX) 49
9.1.2 InterSystems Python Productions (PyProd) 49

9.2 Taking Advantage of InterSystems IRIS's Multi-Model and Analytics Features 50
9.2.1 Some Key Capabilities to Explore as a Power User 50
9.2.2 Mastering Globals: Unlocking High-Performance Data Modeling 50
9.2.3 ObjectScript: The Native Language of InterSystems IRIS 51
9.3 Moving forward 51

List of Figures

Figure 3–1: Python Support with InterSystems IRIS 6
Figure 3–2: Python Processes with InterSystems IRIS 10

1

Welcome, Python Developers

Whether you are a data scientist, backend developer, or integration engineer, Python is likely a key part of your toolkit. With InterSystems IRIS® data platform, you can bring the full power of Python into a high-performance, multi-model data platform without compromising on speed, scalability, or flexibility.

This documentation is a guided journey into using Python with InterSystems IRIS. Whether you are embedding Python directly into InterSystems IRIS logic, building external applications that connect to InterSystems IRIS, or implementing advanced analytics and machine learning, this guide will help you get started and grow your skills.

1.1 What You Will Learn

- Why InterSystems IRIS is a powerful combination for modern data-driven applications.
- How to use the many distinct approaches to Python development with InterSystems IRIS and when to apply each one.
- How to use Python to query InterSystems IRIS like a relational database using tools like DB-API and SQLAlchemy.
- How to build familiar Python applications like dashboards with Streamlit, REST APIs with Flask, and notebooks with Jupyter.
- How to embed Python inside InterSystems IRIS for advanced logic, automation, and hybrid ObjectScript-Python workflows.
- How to create integrations using Python through interoperability productions.
- How to fully utilize InterSystems IRIS's special features like multi-model capabilities, vector search, and the unique power of globals.

1.2 Your Python Journey Starts Here

This guide is structured to support you, whether you are just getting started with InterSystems IRIS Python development or are looking to deepen your expertise. Dive in and unlock what is possible when Python meets InterSystems IRIS.

2

Introduction to InterSystems IRIS Development Using Python

2.1 What Is InterSystems IRIS?

If you are a Python developer, you are probably used to combining multiple tools to build data-driven applications. Perhaps you used a database here, an API layer there, some analytics tools, and a few scripts to connect everything together. InterSystems IRIS® data platform is a data platform that brings all of that into one place without forcing you into a new language or way of thinking.

InterSystems IRIS is a developer-friendly data platform that combines:

- A high-performance, multi-model database capable of storing relational, document, object, key-value, and other types of data.
- Built-in analytics and machine learning support.
- A full interoperability engine for connecting systems and services.
- Native support for Python, providing access to the extensive collection of libraries and the option to run scripts in both embedded and external processes.

And it is all designed to scale, from small applications to mission-critical systems used in healthcare, finance, and more.

2.2 Why InterSystems IRIS is Great for Python Developers

Python developers benefit from using InterSystems IRIS by:

- *Using the tools you already know*: Connect with InterSystems IRIS using SQLAlchemy, DB-API, Flask, Jupyter, Streamlit, and more.
- *Not needing to move data around*: InterSystems IRIS minimizes the need for complex extract, transform, and load (ETL) pipelines by letting you run analytics and Python logic directly on live operational data.
- *Using data flexibly through multi-model*: Store and access data as SQL tables, JSON documents, objects, or even multi-dimensional globals all in one place.

- *Utilizing a built-in Python engine*: You can embed Python directly into InterSystems IRIS logic or call InterSystems IRIS from your external Python applications.
- *Creating interoperability productions easily*: InterSystems IRIS includes tools to connect to other systems, transform data, and orchestrate workflows without needing a separate integration platform.

2.3 What You Can Build with InterSystems IRIS

With InterSystems IRIS and Python, you can build the following:

- REST APIs and microservices.
- Real-time dashboards and data applications.
- Machine learning pipelines.
- Generative AI-powered applications.
- System integrations and automations.
- Scalable, analytics-driven applications.

You do not have to learn a whole new ecosystem to get started. If you know Python, you already have the skills to build with InterSystems IRIS. This platform just gives you more power, performance, and flexibility to do it all in one place.

3

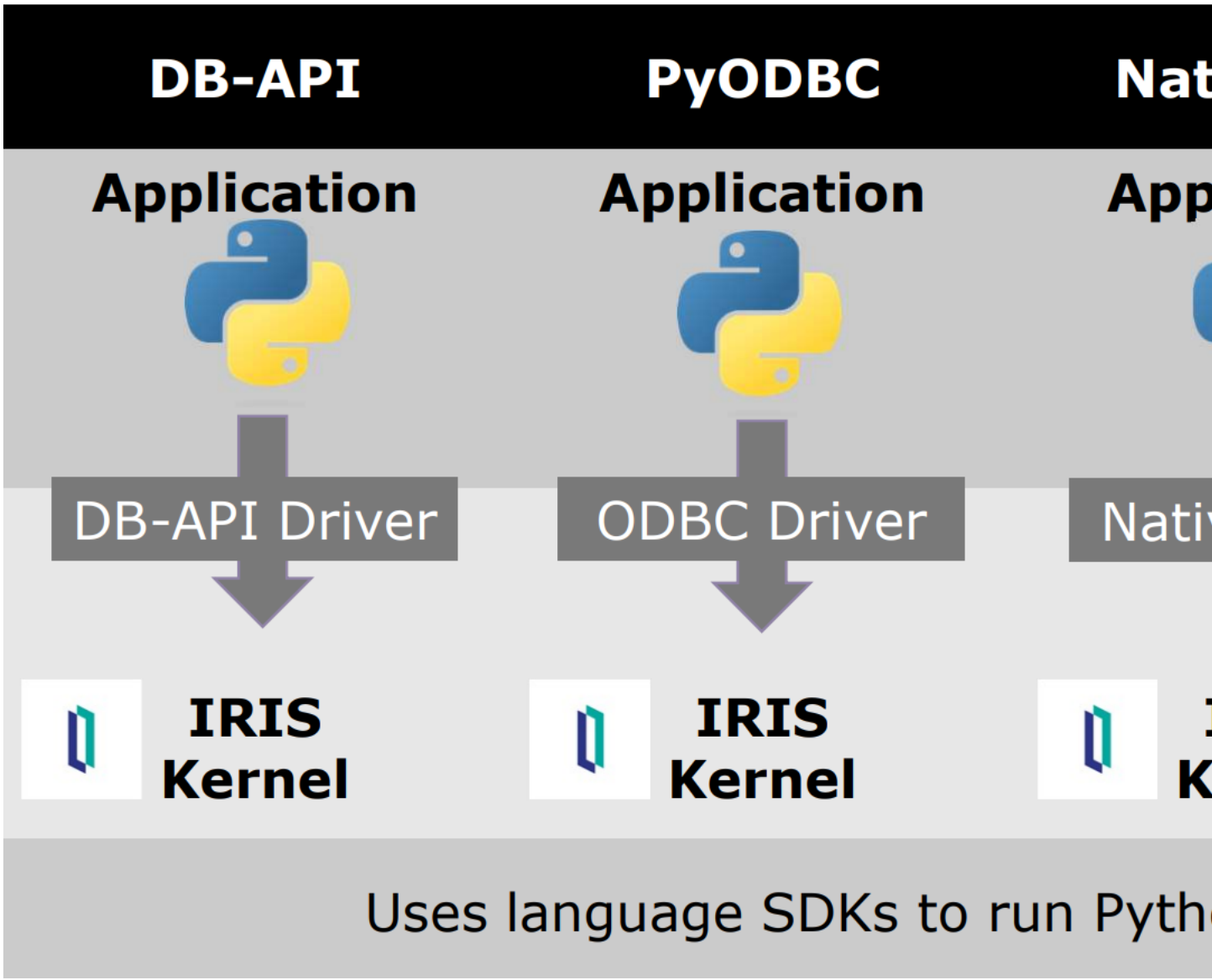
Deciding How to Use InterSystems IRIS with Python

InterSystems IRIS offers flexible and powerful ways to integrate Python into your data workflows, either via external connections (APIs, gateways, and SDKs) or within the kernel (Embedded Python). This document explores the different options for using Python with InterSystems IRIS and helps you choose the one that is appropriate for your needs.

3.1 Python Options

Not every application is the same. Each application requires features and capabilities specific to its needs. InterSystems IRIS offers many options in the ways you can implement your applications with Python depending on what you are looking for. This document summarizes the ways of using Python with InterSystems IRIS and highlights some of the use cases for each of them.

Figure 3–1: Python Support with InterSystems IRIS



3.1.1 Client-Server Options

The four client-server options use language SDKs that run Python outside of the InterSystems IRIS process.

1. *DB-API*: Implements a PEP 249–compliant direct interface with InterSystems databases through the DB-API driver.
2. *pyODBC*: Establishes an Open Database Connectivity (ODBC) connection to InterSystems IRIS through the ODBC driver.
3. *Native API*: Provides direct access to many InterSystems IRIS features from Python through the Native driver.
4. *Python Gateway*: Calls out to an external language server.

3.1.2 Embedded Python

With Embedded Python, Python runs in the same process as InterSystems IRIS. Embedded Python provides the lowest latency out of all these options.

3.2 DB-API

The InterSystems IRIS Python DB-API driver provides a standards-compliant interface for accessing InterSystems IRIS data from external Python applications. It is based on the widely adopted Python DB-API 2.0 (PEP 249), making it easy for developers to work with InterSystems IRIS using familiar Python database interaction patterns.

This driver comes bundled as part of the InterSystems IRIS Python SDK Kit and is distributed via PyPi: `intersystems-irispython`. It enables client-side access to InterSystems IRIS, connecting over the network to interact with data in a relational, SQL-like way.

Note: If you are using DB-API and the client is on same machine as InterSystems IRIS, Python and InterSystems IRIS communicate using shared memory by default, for greater performance. This option can be turned off if you need Python to communicate with InterSystems IRIS using a secure connection, or to simulate connecting from another machine via TCP/IP. See Creating a Connection Object for details.

Note: The InterSystems IRIS Python SDK Kit and Embedded Python both require you to import an `iris` module. Though the modules have some methods with similar names, they have separate APIs and are not interchangeable. DB-API uses the **connect()** method of `iris` module that comes with the InterSystems IRIS Python SDK Kit to create a connection object. For information, see Using the Python DB-API.

3.2.1 When to Use the DB-API Driver

- You are developing external Python applications (for example, data processing, analytics, or web services) that need to connect to InterSystems IRIS over a standard interface.
- You prefer or require working entirely with Python, outside of the InterSystems IRIS environment.
- You want to leverage the standard Python database workflow, including connection objects, cursors, and SQL execution.
- You are integrating InterSystems IRIS with Python tools and frameworks that expect DB-API compatibility (for example, Pandas, SQLAlchemy, ORMs, Flask, and Streamlit).

For more information, see Using Python DB-API.

3.3 pyODBC

pyODBC is a Python module that provides a bridge between Python and databases via the ODBC standard. It implements the Python DB-API 2.0 specification and allows applications to connect to InterSystems IRIS using ODBC drivers.

Important: While it offers broad compatibility and has historically been used in many Python database workflows, pyODBC is no longer the recommended approach for connecting to InterSystems IRIS in most cases. Instead, use the DB-API driver.

3.3.1 When to Use pyODBC

- You need to connect to older versions of InterSystems IRIS (prior to version 2022.1) that do not support the Python DB-API driver.
- You are working in an environment where ODBC is already in place and integration depends on existing Data Source Names (DSNs).

- Your workflow depends on a low-level, ODBC-based architecture, such as certain legacy reporting or business intelligence tools that are being accessed from Python.

For more information, see ODBC Support for Python and Node.js.

3.4 Native API

The InterSystems IRIS Native API allows Python applications to interact directly with InterSystems IRIS through low-level access to its core data structures—specifically, globals, the high-performance, multi-dimensional arrays used internally by InterSystems IRIS. It also lets you access InterSystems IRIS class methods from Python.

This API is part of the InterSystems IRIS Python SDK Kit, which is distributed via PyPi: `intersystems-irispython`. It enables external Python code to connect to InterSystems IRIS over the network and perform non-relational operations without using SQL. It offers fine-grained control over data and is well-suited for advanced or performance-critical applications.

Note: If you are using the Native API and the client is on same machine as InterSystems IRIS, Python and InterSystems IRIS communicate using shared memory by default, for greater performance. This option can be turned off if you need Python to communicate with InterSystems IRIS using a secure connection, or to simulate connecting from another machine via TCP/IP. See Creating a Connection in Python for details.

Note: The InterSystems IRIS Python SDK Kit and Embedded Python both require you to import an `iris` module. Though the modules have some methods with similar names, they have separate APIs and are not interchangeable. For information on the `iris` module used by the Native SDK, see Native SDK for Python Quick Reference.

3.4.1 When to Use the Native API

- You need direct access to InterSystems IRIS globals, bypassing SQL and object layers.
- You are building high-performance or low-latency data processing systems that benefit from schema-less access.
- You are developing Python applications that run outside of the InterSystems IRIS environment.
- Your use case involves non-relational data models, such as hierarchical structures or key-value stores.

For more information, see Introduction to the Native SDK for Python.

3.5 Python Gateway

The Python Gateway, also known as the External Language Server, enables InterSystems IRIS to call out to Python code running externally, reversing the usual client-server relationship. Instead of Python initiating the connection to InterSystems IRIS, InterSystems IRIS becomes the caller, invoking Python code that resides on a separate system or process. This method establishes a communication bridge from InterSystems IRIS to Python, making it possible to integrate Python logic—such as a machine learning models, data processing routines, or specialized computations—into InterSystems IRIS workflows.

The gateway runs as a separate service and communicates with InterSystems IRIS over a defined protocol, allowing Python to be part of business logic, orchestration, and process automation within the InterSystems IRIS environment.

3.5.1 When to Use the Python Gateway

- You want InterSystems IRIS to initiate execution of external Python code that lives outside the database processes.
- You are integrating existing Python applications, scripts, or services into InterSystems IRIS business logic.
- Embedded Python is not available in your deployment, but Python functionality is still required.
- Your use case involves external systems or APIs that are best handled by Python but must be coordinated from within InterSystems IRIS.
- You want to build or extend interoperability productions using Python components.

For more information, see Working with External Languages.

3.5.2 PEX Framework: A Key Use Case for the Python Gateway

The Production EXtension (PEX) framework is an important example of how the Python Gateway architecture is used in practice. PEX allows you to develop interoperability productions—InterSystems IRIS workflows that integrate systems with different message formats and protocols—using external languages such as Python. With PEX, you can implement business services, processes, and adapters in Python that run as separate services connected via the gateway. These Python components are invoked at runtime and communicate with other production elements through the PEX messaging system, enabling seamless integration with InterSystems IRIS interoperability productions.

For more information, see Introduction to the PEX Framework.

3.6 Embedded Python

In the client-server setup, Python and InterSystems IRIS run in separate processes. This means that each request between them must travel across a network boundary. These requests introduce latency, require serialization, and prevent tight integration with InterSystems IRIS-specific features.

In contrast, with Embedded Python, the Python runtime and the InterSystems IRIS runtime are contained in same process. This tight integration means faster database access and streamlined communication between Python and ObjectScript, the native InterSystems procedural programming language.

Figure 3–2: Python Processes with InterSystems IRIS

The benefits of Embedded Python are:

- *Performance*: There is no need to serialize data between InterSystems IRIS and Python.
- *Simplicity*: Seamlessly integrate Python with InterSystems IRIS, and deploy your Python code together with your ObjectScript code.
- *Security*: There is no need to open any additional ports for communication between InterSystems IRIS and Python. You can leverage the InterSystems native security model.
- *Scalability*: Utilize InterSystems IRIS's ECP and sharding features to easily scale your applications.

You can run Embedded Python in one of two basic modes: InterSystems IRIS running inside the Python process or Python running inside the InterSystems IRIS process.

If you are ready to move beyond basic Python database access and fully embrace what InterSystems IRIS has to offer, Embedded Python is the path forward.

Note: The InterSystems IRIS Python SDK Kit and Embedded Python both require you to import an `iris` module. Though the modules have some methods with similar names, they have separate APIs and are not interchangeable. For information on the `iris` module used by Embedded Python, see InterSystems IRIS Python Module Reference.

Note: Virtual environments are not currently supported for Embedded Python, as it uses the specific Python executable specified in the Embedded Python configuration. Client-side Python code can be deployed in a virtual environment, and you can have multiple virtual environments on the same machine that connects to the same InterSystems IRIS process.

3.6.1 InterSystems IRIS Running Inside the Python Process

In this mode, you use the command **irispython** to call in to InterSystems IRIS. By running `irispython myscript.py` from the command line, your Python script runs in the same process with InterSystems IRIS, while your Python code remains separate from any ObjectScript code. This code separation allows you to use all your customary Python development practices: such as debuggers, linters, and syntax coloring tools.

3.6.2 Python Running Inside the InterSystems IRIS Process

In this mode, InterSystems IRIS calls out to Python. There are several ways to initiate Python from InterSystems IRIS, each with specific use cases, for example:

- Write a method in an InterSystems IRIS class using the keyword `[ Language = python ]`. This is useful when you have an existing InterSystems IRIS class and you want to add a simple method written in Python.
- Use the **Import()** method of the %SYS.Python class in InterSystems IRIS. This is useful when you want to import a Python module from ObjectScript in order to perform a well-defined task.
- Launch the Python shell from the Terminal using the **Shell()** method of the %SYS.Python class. This is useful for testing a few lines of Python code interactively from within the InterSystems IRIS environment.

For larger-scale Python development, InterSystems recommends calling in to InterSystems IRIS from a .py file using **irispython**.

Note: When Python is running within InterSystems IRIS, it is operating in a environment with multiple processes, users, home directories, and permissions. This results in added complexity that can sometimes make it more difficult to diagnose an issue when using Embedded Python.

3.7 Choosing the Right Path for You

It is important to choose the right Python option when working with InterSystems IRIS to fully take advantage of each approach. Understanding the nuances of each option can help determine which path to take. Use the following as resources to help navigate the broad Python support system that InterSystems IRIS provides.

3.7.1 Python Support Capabilities

Use Case	DB-API or pyODBC	Native API	Python Gateway	Embedded Python
Client Applications	Yes	Yes	No	No
SQL Stored Procedures, Functions, Triggers	No	No	Possible	Yes
Augmenting existing InterSystems IRIS Classes	No	No	No	Yes
Interoperability	No	No	Yes (PEX)	Possible
Manipulating Globals	No	Yes	Yes	Yes

3.7.2 Python Uses Cheat Sheet

What You Are Building	Recommended Python Method	Why This Works Well
REST APIs, dashboards, client applications, or relational database access requiring SQL calls (for small amounts of data)	*DB-API or pyODBC*	Familiar SQL-based access; great for lightweight, structured data interactions
Applications needing direct access to InterSystems IRIS globals or non-relational data	*Native API*	Offers low-level access to hierarchical and multi-dimensional data structures
InterSystems IRIS logic that needs to call external Python code	*Python Gateway (PEX)*	Allows InterSystems IRIS to trigger Python scripts or models externally; great for interoperability productions
High-performance, data-intensive logic close to the database; importing a third-party Python module for use in an existing ObjectScript application	*Embedded Python*	Runs inside the InterSystems IRIS kernel; lowest latency and tightest integration with the data; makes it possible to use popular Python packages from ObjectScript

4

Use InterSystems IRIS as a Relational Database with DB-API

As a Python developer, you are likely familiar with querying databases using SQL. InterSystems IRIS® data platform supports this workflow seamlessly, allowing you to treat it like a high-performance relational database. However, if you truly want to work with relational data using Python's object-oriented nature, you can use an object-relational mapping (ORM), which converts data representation between a relational database and an object-oriented programming language.

This section introduces two common Python tools that can be used with InterSystems IRIS:

- DB-API: A lightweight, direct SQL interface.
- SQLAlchemy: A powerful ORM and SQL toolkit.

The easiest way to get started is to download and install the InterSystems implementation of SQLAlchemy, which includes the InterSystems IRIS Python SDK Kit as a dependency. The Python SDK Kit includes both the DB-API driver and the Native SDK for Python. This gives you everything you need to start working with InterSystems IRIS from Python.

4.1 Prerequisites

This exercise and the related exercises in this section require the DB-API driver and sometimes SQLAlchemy.

The following command installs both SQLAlchemy (`sqlalchemy-intersystems-iris)` and the DB-API driver (`intersystems-irispython`):

```
pip install sqlalchemy-intersystems-iris
```

4.2 DB-API: Direct SQL Access

The `iris` module provides a PEP 249-compliant interface for executing raw SQL queries. DB-API is the standard interface to interact with any relational backend. It is ideal for lightweight scripts, data access layers, and quick prototyping.

Note: There are multiple `iris` modules, each with their own APIs. This section focuses on the DB-API approach, which is used for external Python applications that connect to InterSystems IRIS.

For complete documentation on the InterSystems implementation of DB-API, including InterSystems-specific extensions, see Using Python DB-API.

4.3 Establishing a DB-API Connection

To establish a connection to an InterSystems IRIS instance using DB-API, use the `iris.connect()` method. This code creates a connection to the InterSystems IRIS instance and opens a cursor for executing SQL commands.

Python

```
import iris

# Replace with your connection details
connection_string = "localhost:1972/USER"
username = "_system"
password = "SYS"
connection = iris.connect(connection_string, username, password)

cursor = connection.cursor()
```

Note: In `connection_string`, `1972` is the port number while `USER` is the namespace that you connect to. Change these to match your specific needs, as well.

Remember to close the cursor and connection when you are done:

Python

```
cursor.close()
connection.close()
```

4.4 Executing a SQL Query

Once connected, you can execute SQL queries using the cursor object. You can then retrieve the results from the query using the methods **fetchone()**, **fetchmany()**, **fetchall()**, or **scroll()**.

This example uses the Sample.Person class from the Samples-Data repository on GitHub: https://github.com/intersystems/Samples-Data.

Python

```
cursor.execute("SELECT * FROM Sample.Person WHERE Age >= 50")

row
=
cursor.fetchone()
while row
is
not
None:
    print(row[:])
    row
=
cursor.fetchone()
```

In the above example, **fetchone()** returns a pointer to the next row, or `None` if no more data is available.

4.5 Parameters

Parameters help prevent SQL injections and can make your queries more flexible. With DB-API, you can specify both positional and named parameters to extend your queries. Pass the parameters along with the SQL statement to the cursor to execute them.

Note: The Sample.Person class on GitHub is created with randomized data. To get the parameter examples below to run, find an ID and Name from your sample data.

4.5.1 Positional Parameters

Positional parameters match the question marks in the SQL statement with the arguments in the parameters list by position.

Python

```
sql = "SELECT * FROM Sample.Person WHERE ID = ? and Name = ?"
params = [1, 'Doe,John Q.']
cursor.execute(sql, params)
result = cursor.fetchone()
row = result[:]
print(row)
```

4.5.2 Named Parameters

Named parameters match the `:argument` variables in the SQL statement with the arguments in the parameters dictionary by keyword.

Python

```
sql = "SELECT * FROM Sample.Person WHERE ID = :id and Name = :name"
params = {'id' : '1', 'name' : 'Doe,John Q.'}
cursor.execute(sql, params)
result = cursor.fetchone()
row = result[:]
print(row)
```

For more documentation on DB-API, see Using the Python DB-API.

5

Access Relational Data as Objects with SQLAlchemy

While the previous exercise used DB-API to use InterSystems IRIS as a relational database from Python, this exercise shows you how to use relational data as objects using SQLAlchemy.

5.1 Prerequisites

Before starting this exercise, make sure that you have installed SQLAlchemy and DB-API using the following command:

```
pip install sqlalchemy-intersystems-iris
```

See the DB-API exercise for a brief description of these tools.

5.2 SQLAlchemy: SQL Toolkit and ORM

SQLAlchemy is a powerful SQL toolkit and ORM tool built on top of DB-API. It provides a higher-level abstraction over SQL, allowing you to define tables as Python classes and interact with them using ORM. It is ideal for larger applications that require cleaner, more maintainable code and helps you avoid raw SQL when possible.

SQLAlchemy has two major components:

1. Core—for executing raw SQL and building SQL expressions.
2. ORM—for mapping Python classes to database tables and managing transactions.

5.3 Establishing an Engine Connection

To connect to InterSystems IRIS using SQLAlchemy, use the `create_engine()` function with the appropriate dialect and connection string. The dialect is specified before the `://` in the `DATABASE_URL`. The engine is the central source of connection and is used for both the Core and ORM.

Python

```
from sqlalchemy import create_engine

# Replace with your credentials and connection information
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

engine = create_engine(DATABASE_URL, echo=True) # Set echo=True to see the SQL queries being executed
 in the terminal
```

Just like in DB-API, you can execute simple static queries via the `.execute()` method:

Python

```
# SQL statements get wrapped in text sequences
from sqlalchemy import text

# Connect to the database and execute a static query
with engine.connect() as conn:
    query = text("SELECT * FROM Sample.Person WHERE ID = 1")
    result = conn.execute(query)
    row = result.fetchone()
    print(row)
```

5.4 Transactions and Sessions

InterSystems IRIS supports robust transaction management through SQLAlchemy, whether you are using the Core or ORM interface. Through this, you have control over transaction states, commits, rollbacks, and much more.

5.4.1 engine.begin()

The `engine.begin()` method is a context manager that starts a transaction. It starts a transaction block that automatically commits when the block exits without an error.

Python

```
# engine.begin starts a transaction block with an auto-commit
with engine.begin() as conn:
    result = conn.execute(text("SELECT Name, Age FROM Sample.Person"))
    for row in result:
        print(f"Name: {row.Name}\nAge: {row.Age}\n")
```

5.4.2 engine.connect()

The `engine.connect()` starts a transaction block that automatically rolls back when the block exists. To save any changes made in the transaction, you must call to `conn.commit()` to commit your changes.

Python

```
# engine.connect starts a transaction block with an automatic rollback
with engine.connect() as conn:
    conn.execute(text("INSERT INTO Sample.Person (Name,Home_Street,Home_City,SSN) \
    VALUES('Simpson,Homer J.','742 Evergreen Terrace','Springfield','000-00-0000')"))
    conn.commit()  # Need to explictly commit to save changes
```

The following example, run from the Python shell, illustrates this concept. The first insert is committed to the database, while the second insert is rolled back.

Terminal

```
>>> with engine.connect() as conn:
...     conn.execute(text("INSERT INTO Sample.Person (Name,Home_Street,Home_City,SSN) \
...     VALUES('Simpson,Homer J.','742 Evergreen Terrace','Springfield','000-00-0000')"))
...     conn.commit()
...
2026-03-26 12:07:01,499 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 12:07:01,501 INFO sqlalchemy.engine.Engine INSERT INTO Sample.Person
(Name,Home_Street,Home_City,SSN) VALUES('Simpson,Homer J.','742 Evergreen
Terrace','Springfield','000-00-0000')
2026-03-26 12:07:01,502 INFO sqlalchemy.engine.Engine [generated in 0.00296s] ()
2026-03-26 12:07:01,530 INFO sqlalchemy.engine.Engine COMMIT
>>> with engine.connect() as conn:
...     conn.execute(text("INSERT INTO Sample.Person (Name,Home_Street,Home_City,SSN) \
...     VALUES('Simpson,Homer J.','742 Evergreen Terrace','Springfield','000-00-0001')"))
...
2026-03-26 12:08:04,754 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 12:08:04,754 INFO sqlalchemy.engine.Engine INSERT INTO Sample.Person
(Name,Home_Street,Home_City,SSN) VALUES('Simpson,Homer J.','742 Evergreen
Terrace','Springfield','000-00-0001')
2026-03-26 12:08:04,755 INFO sqlalchemy.engine.Engine [generated in 0.00132s] ()
2026-03-26 12:08:04,763 INFO sqlalchemy.engine.Engine ROLLBACK
```

5.4.3 Session(engine)

Then `Session` object provides a high-level interface for managing transactions and interacting with ORM objects. It establishes a conversation between the database and your code, giving you fine-grained control over commits, rollbacks, and object states.

Python

```
# Session starts an ORM session with fine-grained commit/rollback controls
from sqlalchemy.orm import Session

print("Listing rows where Age > 50:")
stmt = text("SELECT Name, Age FROM Sample.Person WHERE Age > :min_age ORDER BY Age, Name")
with Session(engine) as session:
    result = session.execute(stmt, {"min_age": 50})
    for row in result:
        print(f"Name: {row.Name}\nAge: {row.Age}\n")
```

5.5 SQLAlchemy Core: Fine-Grained SQL Control

SQLAlchemy Core provides a schema-first, lower-level approach for building and executing SQL statements using Python constructs. It gives you precise control over SQL generation and execution making it ideal for:

- Complex or dynamic SQL queries.
- Performance-sensitive applications.
- Developers who prefer SQL-like control without raw strings.

5.5.1 Defining Tables with SQL Alchemy Core

You can define tables using the `Table()` and `Column()` constructors and then execute SQL statements using the `select()`, `insert()`, `update()`, and `delete()` functions. Remember to first create the table metadata. This allows SQLAlchemy to keep track of the table structure and determine whether it needs to create the table in the database when calling `create_all()` on the metadata object.

Python

```
from sqlalchemy import Column, MetaData, Table
from sqlalchemy.sql.sqltypes import Integer, VARCHAR
from sqlalchemy import create_engine

# Replace with your credentials and connection information
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

engine = create_engine(DATABASE_URL, echo=True)

# Create a table metadata
metadata = MetaData()

# Define the table structure
demo_table = Table(
    "demo_table",
    metadata,
    Column("id", Integer, primary_key = True, autoincrement = True),
    Column("value", VARCHAR(50)),
)

# Create the table from the metadata
metadata.create_all(engine)

# Insert sample data
with engine.connect() as conn:
    conn.execute(
        demo_table.insert(),
        [
            {"id": 1, "value": "Test"},
            {"id": 2, "value": "More"},
        ],
    )
    conn.commit()
    result = conn.execute(demo_table.select()).fetchall()
    print("result:", result)
```

Note: Tables created using this method will be seen as `SQLUser.demo_table` in the InterSystems IRIS Management Portal.

5.6 SQLAlchemy ORM: Object-Oriented Data Access

The ORM layer abstracts away the relational structure of your data allows you to define tables as Python classes. In this way, you work with data in a more “Pythonic” way as you and interact with data using traditional Python objects.

It is ideal for:

- Create, read, update, delete (CRUD) operations.
- Business logic encapsulation.
- Applications that benefit from abstraction.

5.6.1 Creating ORM Models

ORM allows you to define database tables as Python classes. Each class represents a table, and each attribute represents a column. This abstraction simplifies database operations and integrates well with Python applications. All tables inherit from the `Base` class (which itself inherits from `DeclarativeBase`).

Python

```
from typing import List, Optional
from sqlalchemy import String, ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "user_account"

    # 'id' column is the primary key of the table
    id: Mapped[int] = mapped_column(primary_key=True)

    # 'name' column has a maximum string length of 30 characters
    name: Mapped[str] = mapped_column(String(30))

    # 'fullname' column is optional, so it can be NULL in the database
    fullname: Mapped[Optional[str]]

    # String representation of the object; useful for debugging
    def __repr__(self) -> str:
        return f"User(id = {self.id!r}, name = {self.name!r}, fullname = {self.fullname!r})"
```

The general format for defining an object is:

Python

```
columnName: Mapped[type] = mapped_column(db.Type, arguments)
```

5.6.2 Saving Data

To save data using ORM, create an instance of the model class and add it to a session. Then commit the session to persist the changes to the database.

Python

```
Base.metadata.create_all(engine) # Create the table if it does not exist

with Session(engine) as session:
    # Create new instances of the User class
    frodo = User(
        name = "frodo",
        fullname = "Frodo Baggins"
    )
    samwise = User(
        name = "samwise",
        fullname = "Samwise Gamgee"
    )
    pippin = User(
        name = "pippin",
        fullname = "Pippin Took")

    # Add all three new User objects to the session (prepares them to be inserted)
    session.add_all([frodo, samwise, pippin])

    # Commit the session; writes the changes (inserts) to the database
    session.commit()
```

5.6.3 Retrieving Data

To retrieve data using ORM, use the `Session` object to query the model class. You can use filters and other query methods to refine your search results.

Python

```
from sqlalchemy import select  # 'select' is used to build SQL SELECT queries

with Session(engine) as session:
    # Print users with specific names
    stmt = select(User).where(User.name.in_(["frodo", "samwise"]))

    # 'scalars()' extracts the actual User objects from the result
    for user in session.scalars(stmt):
        print(user)
```

5.6.4 Deleting Data

To delete records using SQLAlchemy ORM, you first query the object you want to remove, then pass it to the session's `delete()` method. Finally, commit the session to apply the change to the database.

Python

```
with Session(engine) as session:
    # List of IDs to delete
    ids_to_delete = [1, 2, 3]

    for user_id in ids_to_delete:
        user = session.get(User, user_id)
        if user:
            session.delete(user)
    session.commit()
```

5.6.5 Running the Script

Putting all of the code in a script, orm.py, and running it yields the following output:

Terminal

```
C:\Users\Test>python orm.py
2026-03-26 17:36:47,565 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 17:36:47,569 INFO sqlalchemy.engine.Engine SELECT count(*) AS count_1
FROM "INFORMATION_SCHEMA"."TABLES"
WHERE "INFORMATION_SCHEMA"."TABLES"."TABLE_SCHEMA" = ? AND "INFORMATION_SCHEMA"."TABLES"."TABLE_NAME"
 = ?
2026-03-26 17:36:47,570 INFO sqlalchemy.engine.Engine [generated in 0.00075s] ('SQLUser', 'user_account')
2026-03-26 17:36:47,572 INFO sqlalchemy.engine.Engine COMMIT
2026-03-26 17:36:47,574 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 17:36:47,575 INFO sqlalchemy.engine.Engine INSERT INTO user_account (name, fullname) VALUES
 (?, ?)
2026-03-26 17:36:47,575 INFO sqlalchemy.engine.Engine [generated in 0.00022s] ('frodo', 'Frodo Baggins')
2026-03-26 17:36:47,576 INFO sqlalchemy.engine.Engine INSERT INTO user_account (name, fullname) VALUES
 (?, ?)
2026-03-26 17:36:47,576 INFO sqlalchemy.engine.Engine [cached since 0.001451s ago] ('samwise', 'Samwise
 Gamgee')
2026-03-26 17:36:47,577 INFO sqlalchemy.engine.Engine INSERT INTO user_account (name, fullname) VALUES
 (?, ?)
2026-03-26 17:36:47,577 INFO sqlalchemy.engine.Engine [cached since 0.002317s ago] ('pippin', 'Pippin
 Took')
2026-03-26 17:36:47,578 INFO sqlalchemy.engine.Engine COMMIT
2026-03-26 17:36:47,579 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 17:36:47,580 INFO sqlalchemy.engine.Engine SELECT user_account.id, user_account.name,
user_account.fullname
FROM user_account
WHERE user_account.name IN (?, ?)
2026-03-26 17:36:47,580 INFO sqlalchemy.engine.Engine [generated in 0.00028s] ('frodo', 'samwise')
User(id = 1, name = 'frodo', fullname = 'Frodo Baggins')
User(id = 2, name = 'samwise', fullname = 'Samwise Gamgee')
User(id = 4, name = 'frodo', fullname = 'Frodo Baggins')
User(id = 5, name = 'samwise', fullname = 'Samwise Gamgee')
2026-03-26 17:36:47,604 INFO sqlalchemy.engine.Engine ROLLBACK
2026-03-26 17:36:47,606 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-26 17:36:47,607 INFO sqlalchemy.engine.Engine SELECT user_account.id AS user_account_id,
user_account.name AS user_account_name, user_account.fullname AS user_account_fullname
FROM user_account
WHERE user_account.id = ?
2026-03-26 17:36:47,607 INFO sqlalchemy.engine.Engine [generated in 0.00026s] (1,)
2026-03-26 17:36:47,624 INFO sqlalchemy.engine.Engine DELETE FROM user_account WHERE user_account.id
```

```
= ?
2026-03-26 17:36:47,624 INFO sqlalchemy.engine.Engine [generated in 0.00028s] (1,)
2026-03-26 17:36:47,641 INFO sqlalchemy.engine.Engine SELECT user_account.id AS user_account_id,
user_account.name AS user_account_name, user_account.fullname AS user_account_fullname
FROM user_account
WHERE user_account.id = ?
2026-03-26 17:36:47,641 INFO sqlalchemy.engine.Engine [cached since 0.03398s ago] (2,)
2026-03-26 17:36:47,642 INFO sqlalchemy.engine.Engine DELETE FROM user_account WHERE user_account.id
= ?
2026-03-26 17:36:47,642 INFO sqlalchemy.engine.Engine [cached since 0.01814s ago] (2,)
2026-03-26 17:36:47,643 INFO sqlalchemy.engine.Engine SELECT user_account.id AS user_account_id,
user_account.name AS user_account_name, user_account.fullname AS user_account_fullname
FROM user_account
WHERE user_account.id = ?
2026-03-26 17:36:47,643 INFO sqlalchemy.engine.Engine [cached since 0.03637s ago] (3,)
2026-03-26 17:36:47,644 INFO sqlalchemy.engine.Engine DELETE FROM user_account WHERE user_account.id
= ?
2026-03-26 17:36:47,644 INFO sqlalchemy.engine.Engine [cached since 0.02012s ago] (3,)
2026-03-26 17:36:47,645 INFO sqlalchemy.engine.Engine COMMIT
```

6

Build an Interactive Application with Streamlit

Once you are comfortable querying InterSystems IRIS® data platform using DB-API and SQLAlchemy, you can start building interactive applications that work with your data. This section introduces how to use Streamlit to explore and visualize data from InterSystems IRIS interactively and build web applications and dashboards

6.1 Prerequisites

Before starting this exercise, make sure that you have installed SQLAlchemy and DB-API using the following command:

```
pip install sqlalchemy-intersystems-iris
```

See the DB-API exercise for a brief description of these tools.

You also need to install Streamlit:

```
pip install streamlit
```

This particular application uses Pandas to help parse data more easily and Plotly Express to create interactive visualizations. Make sure to install them as well for this application:

```
pip install pandas plotly
```

You should also have access to a running InterSystems IRIS instance and valid connection credentials (host, port, username, password, namespace).

6.2 Building Interactive Apps with Streamlit and InterSystems IRIS

Streamlit is a Python framework that allows developers to build interactive web applications with minimal code. You do not need to know HTML, CSS, or JavaScript. When paired with InterSystems IRIS and SQLAlchemy, it becomes a powerful tool for creating data-driven dashboards, query interfaces, and lightweight front ends.

This section walks through a complete Streamlit application that connects to InterSystems IRIS, runs SQL queries, visualizes data, and uploads CSV files into the database.

6.3 Building Your First Streamlit InterSystems IRIS App

In this section, you build a fully functional Streamlit application that connects to InterSystems IRIS to query data, create visualizations, and even allow to file uploads to update your InterSystems IRIS database—all using Python.

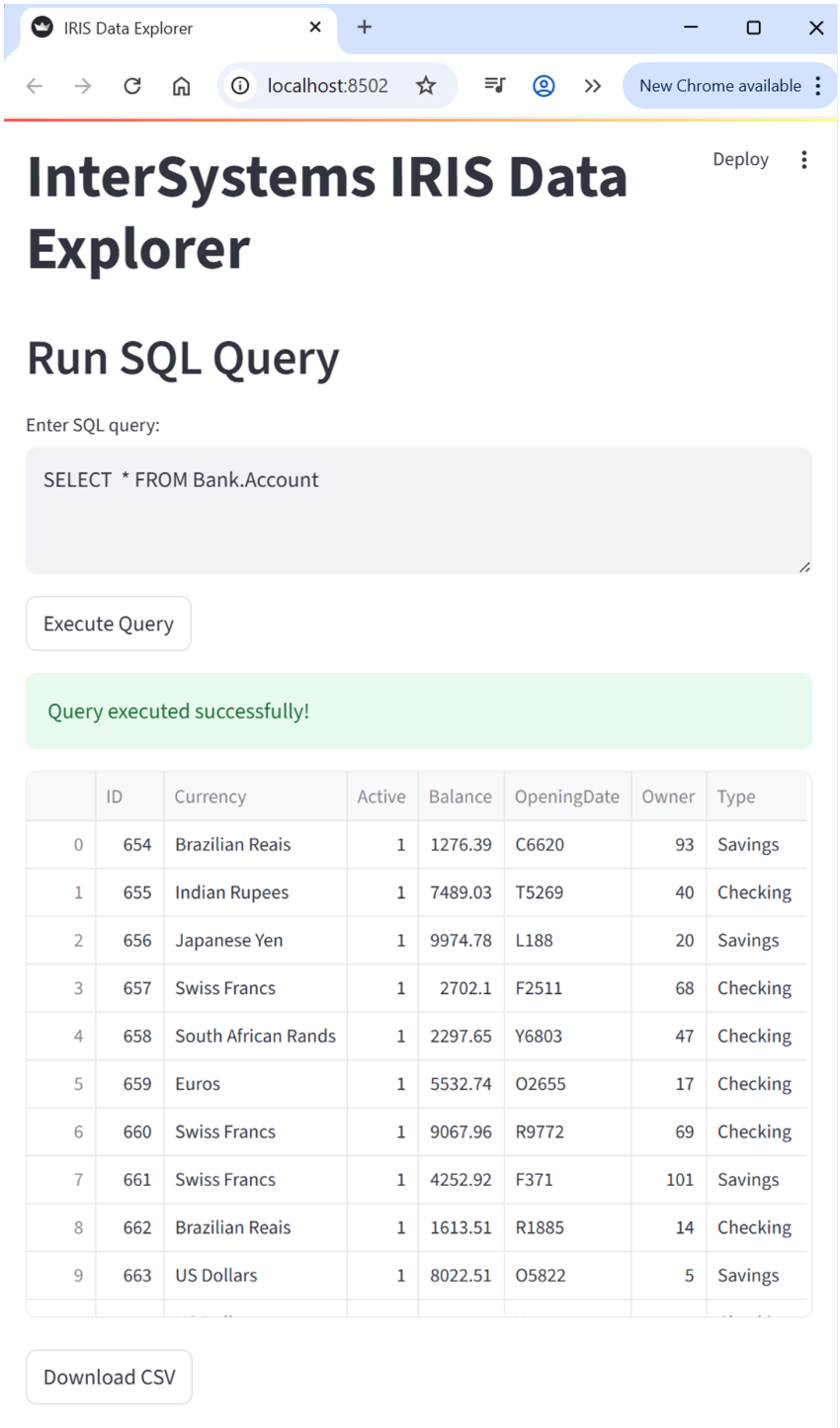
You create a lightweight, interactive web interface where users can:

- Connect to an InterSystems IRIS database and run custom SQL queries.
- View query results in an interactive table.
- Download results as CSV files.
- Visualize numeric data with Plotly charts.
- Upload CSV files and insert their contents into existing InterSystems IRIS tables.

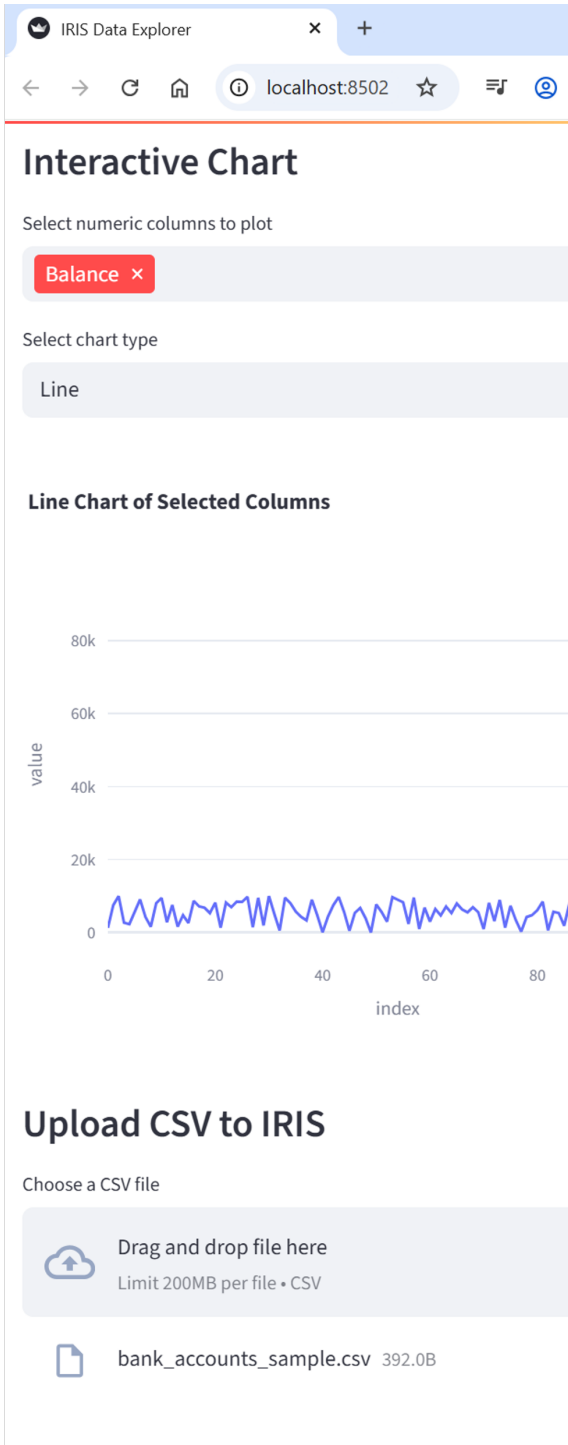
This app is great for data analysts, data engineers, or developers who want a quick and intuitive interface for working with their InterSystems IRIS data, without needing to build a full web front end from scratch.

6.4 What You Build (at a Glance)

Here is what the final Streamlit app looks like:



	ID	Currency	Active	Balance	OpeningDate	Owner	Type
0	654	Brazilian Reais	1	1276.39	C6620	93	Savings
1	655	Indian Rupees	1	7489.03	T5269	40	Checking
2	656	Japanese Yen	1	9974.78	L188	20	Savings
3	657	Swiss Francs	1	2702.1	F2511	68	Checking
4	658	South African Rands	1	2297.65	Y6803	47	Checking
5	659	Euros	1	5532.74	O2655	17	Checking
6	660	Swiss Francs	1	9067.96	R9772	69	Checking
7	661	Swiss Francs	1	4252.92	F371	101	Savings
8	662	Brazilian Reais	1	1613.51	R1885	14	Checking
9	663	US Dollars	1	8022.51	O5822	5	Savings



6.5 Connecting to InterSystems IRIS

To connect to InterSystems IRIS using SQLAlchemy, define a connection string and create an engine. This engine object is reused throughout the app to run queries and insert data and acts like a persistent, reusable pipeline to your database.

Python

```
from sqlalchemy import create_engine

# Replace with your credentials and connection information
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

# Set echo=True to see the SQL queries being excuted in the command line
engine = create_engine(DATABASE_URL, echo=True)
```

For more documentation on using SQLAlchemy with InterSystems IRIS, see InterSystems IRIS and SQLAlchemy.

6.6 Configuring the Streamlit App Interface

Initialize the Streamlit app by setting the page configuration and title. This helps customize the app layout and metadata.

Python

```
import streamlit as st

st.set_page_config(page_title="IRIS Data Explorer")
st.title("InterSystems IRIS Data Explorer")
```

`set_page_config()` allows you to customize the app's layout and metadata.

6.7 Entering Queries

Give users a text box to enter SQL queries. You can provide an example query to help them get started.

Python

```
st.header("Run SQL Query")

# Create a text area where users can enter their SQL query
# The second argument is the default query shown in the box
query = st.text_area("Enter SQL query:", "SELECT TOP 10 * FROM Sample.Person")
```

This lets users interactively explore any part of the database they have access to.

6.8 Executing SQL Queries

When users click the "Execute Query" button, they run the query against your InterSystems IRIS database using `engine.connect()` and store the result in memory using `st.session_state`. The `engine.connect()`creates a connection between Streamlit and your data using SQLAlchemy (see InterSystems IRIS and SQLAlchemy for more details on this connection). Understanding the underlying mechanism in `engine.connect()`is not necessary to complete this application. Just know that you need a way to communicate between Streamlit and InterSystems IRIS. Using `engine.connect()` accomplishes that.

Python

```
# Create a button labeled "Execute Query"
# When clicked, the code inside the if-block runs
if st.button("Execute Query"):
    try:
        # Open a connection to the IRIS database using SQLAlchemy
        with engine.connect() as conn:
            # Use pandas to execute the SQL query and load the result into a DataFrame
            df = pd.read_sql(query, conn)

        st.success("Query executed successfully!")

        # Display the resulting DataFrame as an interactive table in the app
        st.dataframe(df)

        # Save df in session_state
        st.session_state['df'] = df

    except Exception as e:
        st.error(f"Error: {e}")
```

Streamlit reruns your script top-to-bottom on every user interaction. Using `st.session_state` helps you retain data like query results between runs. Without saving `df` into your session state, you run into errors saying that `df` is undefined.

6.9 Displaying Query Results in a Table

Show the query results in an interactive table.

Python

```
if 'df' in st.session_state:
    df = st.session_state['df']

    # Display the table in your Streamlit app
    st.dataframe(df)
```

The `dataframe` command displays an interactive table of your data. It allows you to sort, filter, and scroll through the table, all within the browser.

6.10 Downloading Query Results as CSV Files

Let users export their results as a CSV file through a download button widget.

Python

```
if 'df' in st.session_state:
    csv = st.session_state['df'].to_csv(index=False).encode("utf-8")

    st.download_button(
        label="Download CSV",
        data=csv,
        file_name="results.csv",
        mime="text/csv"
    )
```

In the example above:

- `index=False` removes the DataFrame's index column from the CSV file.
- UTF-8 encoding ensures broad compatibility.
- The MIME type tells the browser that this is a CSV file.

This is useful for offline analysis or sharing results with others.

6.11 Visualizing Data

Plotly Express is a powerful graphing library that integrates well with Streamlit for generating all sorts of charts. By feeding the data you have been extracting into Plotly, you can produce nice visualizations in web browser, all within Streamlit. Regardless of what library you use (matplotlib, ggplot2, seaborn, and so on), you can use InterSystems IRIS and Streamlit to add visualizations to your applications.

6.11.1 Retrieving and Validating Data

Before rendering any chart, we need to:

- Check if query results (`df`) are available.
- Extract only numeric columns (since visualizations depend on numerical data).

Python

```
# Check if a DataFrame from a previous query exists
if 'df' in st.session_state:
    df = st.session_state['df']  # Get the stored DataFrame

    # Identify numeric columns for plotting
    numeric_cols = df.select_dtypes(include='number').columns.tolist()

    # Warn the user if there is nothing numeric to chart
    if df.empty or not numeric_cols:
        st.info("No numeric data available for visualization.")
```

Use `select_dtypes(include='number')` to filter numeric columns, which are required for charts like line, bar, and scatter plots.

6.11.2 User Input—Column and Chart Type Selection

Once numeric data is available, the user can:

- Choose which numeric columns to plot.
- Choose the type of chart they want to render.

Python

```
else:
 st.subheader("Interactive Chart")

 # Multiselect input to choose numeric columns to plot
 cols = st.multiselect(
  "Select numeric columns to plot",
  options=numeric_cols,
  default=[numeric_cols[0]]  # Preselect the first numeric column
 )

 # Dropdown menu to select the chart type
 chart_type = st.selectbox(
  "Select chart type",
  ["Line", "Bar", "Area", "Scatter"]
 )
```

Use `st.multiselect()` to let users plot multiple columns at once. For scatter plots, ensure they choose exactly 2 columns.

Note: The `else` statement comes from the fact that we first checked that there is some numeric data to chart from above.

6.11.3 Generating Chart

Based on the user's input, we create the appropriate chart using Plotly Express.

Python

```
import plotly.express as px

# Only proceed if columns are selected
if cols:
    fig = None

    if chart_type == "Line":
        fig = px.line(df, y=cols, title="Line Chart of Selected Columns")

    elif chart_type == "Bar":
        fig = px.bar(df, y=cols, title="Bar Chart of Selected Columns")

    elif chart_type == "Area":
        fig = px.area(df, y=cols, title="Area Chart of Selected Columns")

    elif chart_type == "Scatter":
        if len(cols) >= 2:
            fig = px.scatter(
                df,
                x=cols[0],  # First selected column as x-axis
                y=cols[1],  # Second selected column as y-axis
                title=f"Scatter Plot: {cols[0]} vs {cols[1]}"
            )
        else:
            st.warning("Select at least 2 columns for Scatter plot")
```

6.11.4 Rendering Chart

Finally, if a figure was successfully created, we render it in the Streamlit app.

Python

```
# Render the Plotly figure inside the streamlit app
if fig:
    st.plotly_chart(fig, use_containter_width=True)
```

6.12 Uploading CSV Data into InterSystems IRIS

Users can upload a CSV file through a Streamlit app and insert its contents into a pre-existing InterSystems IRIS table using SQLAlchemy.

6.12.1 Uploading a CSV

Give users the option to upload a CSV in the Streamlit app.

Python

```
st.subheader("Uplaod CSV to IRIS")
uploaded_file = st.file_loader("Choose a CSV file", type="csv")
```

- `st.file_uploader` allows users to upload files through the user interface.
- The file type is restricted to `.csv` to ensure format consistency.

6.12.2 Previewing the Data

Once the file is uploaded, use pandas to read it and show a preview of it.

Python

```
if uploaded_file:
    csv_df = pd.read_csv(uploaded_file)
    st.dataframe(csv_df.head())
```

- `pandas.read_csv()` parses the file.
- The first few rows of the uploaded file are displayed with `.dataframe()` for review before insertion.

6.12.3 Entering Target Table Name

Ask the user to enter the InterSystems IRIS table name (for example, `Sample.Person`).

Python

```
# Input: full table name in format "Namespace.Table"
full_table_name = st.text_input("Enter target IRIS table (e.g., Bank.Account)")
```

- Users enter the full InterSystems table name, optionally including the namespace (for example, `Bank.Account`).
- If a dot (`.`) is present, it is interpreted as `schema.table`.

6.12.4 Inserting Data into InterSystems IRIS

Now insert the DataFrame into the specified table.

Python

```
if st.button("Insert Data"):
    try:
        # Split schema (namespace) and table name if dot notation is used
        if '.' in full_table_name:
            schema, table_name = full_table_name.split('.', 1)
        else:
            schema = None
            table_name = full_table_name

        # Attempt to insert the data into an existing table
        with engine.begin() as conn:
            csv_df.to_sql(
                name=table_name,
                con=conn,
                if_exists='append',
                index=False,
                schema=schema,
                method='multi'  # Batch insert to InterSystems IRIS
            )

        st.success(f"Successfully inserted data into {full_table_name}")

    except Exception as e:
        st.error(f"Insertion failed: {e}")
```

- The input is parsed to separate the schema (namespace) and table name.
- If no namespace is provided, `schema=None` is passed, and InterSystems IRIS uses the default namespace.
- `engine.begin()` ensures that the operation runs within a transaction context.
- In `to_sql()`:

- *name* is the table name.
- `schema` is the InterSystems IRIS namespace (optional).
- `if_exists='append'` prevents a table override.
- `method='multi'` improves the performance by batching inserts.

Note: The target table must already exist in InterSystems IRIS. This method does not create new tables.

If your table includes an auto-generated `ID` column, you must not try to insert into it directly unless explicitly allowed. To avoid `RowID` conflicts (for example, auto-generated `ID` columns), drop before `ID` inserting.

Python

```
if "ID" in csv_df.columns:
    st.warning("Dropping 'ID' column to let IRIS auto-generate it.")
    csv_df = csv_df.drop(columns=["ID"])
```

6.13 Running Your Application

Once you have finished building your Streamlit app, running it locally is quick and easy.

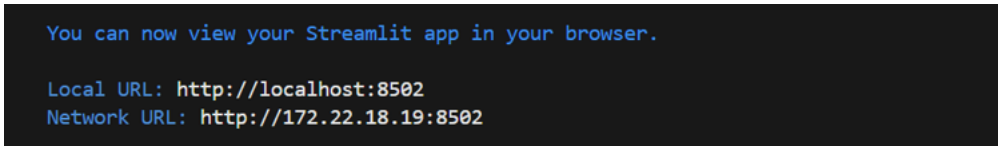
Save your complete code in a file (for example, `app.py`), and then your Streamlit app from the command line:

```
streamlit run app.py
```

If the `streamlit` is not recognized, you can also run it as a Python module:

```
python -m streamlit app.py
```

Your browser should automatically open the app. The command lines display logs, status updates, and the URL to access your app manually. You can now interact with your app live, whether you are running queries, uploading files, or visualizing data.



When you make changes to your `app.py` file:

- Simply save the file, and Streamlit detects any changes.
- By default, the Streamlit app prompts you to rerun it. Click “Always rerun” (in the top-right corner of the browser) for a smoother workflow.

Visit the official Streamlit, pandas, and Plotly documentation to explore more available capabilities.

6.14 Complete Streamlit Code

The following is the entire code for the built Streamlit application.

Python

```
# Initializing Streamlit app
import streamlit as st

import pandas as pd

# Import Plotly Express for interactive charting
import plotly.express as px

# SQLAlchemy Connection to InterSystems IRIS
from sqlalchemy import create_engine

# Connect to IRIS
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

engine = create_engine(DATABASE_URL, echo=True)

st.set_page_config(page_title="IRIS Data Explorer")
st.title("InterSystems IRIS Data Explorer")

st.header("Run SQL Query")

# Create a text area where users can enter their SQL query
# The second argument is the default query shown in the box
query = st.text_area("Enter SQL query:", "SELECT TOP 10 * FROM Sample.Person")

# Create a button labeled "Execute Query"
# When clicked, the code inside the if-block runs
if st.button("Execute Query"):
    try:
        # Open a connection to the IRIS database using SQLAlchemy
        with engine.connect() as conn:
            # Use pandas to execute the SQL query and load the result into a DataFrame
            df = pd.read_sql(query, conn)

        st.success("Query executed successfully!")

        # Display the resulting DataFrame as an interactive table in the app
        st.dataframe(df)

        # Save df in session_state
        st.session_state['df'] = df

    except Exception as e:
        st.error(f"Error: {e}")

if 'df' in st.session_state:
    csv = st.session_state['df'].to_csv(index=False).encode("utf-8")

    st.download_button(
        label="Download CSV",
        data=csv,
        file_name="results.csv",
        mime="text/csv"
    )

# Check if a DataFrame from previous query exists
if 'df' in st.session_state:
    df = st.session_state['df']  # Get the stored DataFrame

    # Identify numeric columns for plotting
    numeric_cols = df.select_dtypes(include='number').columns.tolist()

    # Warn the user if there's nothing numeric to chart
    if df.empty or not numeric_cols:
        st.info("No numeric data available for visualization.")

# Check if df is stored in session_state and visualize
if 'df' in st.session_state:
    df = st.session_state['df']

# Check if the query results DataFrame ('df') exists in Streamlit's session_state
if 'df' in st.session_state:
    df = st.session_state['df']  # Retrieve the DataFrame from session state

    # Extract only numeric columns from the DataFrame for plotting
    numeric_cols = df.select_dtypes(include='number').columns.tolist()

    # If no data is available or there are no numeric columns, notify the user
    if df.empty or not numeric_cols:
```

```
        st.info("No numeric data available for visualization.")
    else:
        # Display a subheader for the chart section
        st.subheader("Interactive Chart")

        # Allow users to select one or more numeric columns to plot
        # The default is the first numeric column
        cols = st.multiselect(
            "Select numeric columns to plot",
            options=numeric_cols,
            default=[numeric_cols[0]]
        )

        # Let users choose the type of chart to generate
        chart_type = st.selectbox(
            "Select chart type",
            ["Line", "Bar", "Area", "Scatter"]
        )

        # Only proceed if at least one column is selected
        if cols:
            fig = None  # Initialize the figure object

            # Generate the appropriate chart based on the selected type
            if chart_type == "Line":
                # Plot a line chart with the selected columns on the y-axis
                fig = px.line(df, y=cols, title="Line Chart of Selected Columns")

            elif chart_type == "Bar":
                # Plot a bar chart with the selected columns on the y-axis
                fig = px.bar(df, y=cols, title="Bar Chart of Selected Columns")

            elif chart_type == "Area":
                # Plot an area chart with the selected columns on the y-axis
                fig = px.area(df, y=cols, title="Area Chart of Selected Columns")

            elif chart_type == "Scatter":
                # Scatter plot requires at least 2 numeric columns
                if len(cols) >= 2:
                    # Use first column as x-axis and second as y-axis
                    fig = px.scatter(
                        df,
                        x=cols[0],
                        y=cols[1],
                        title=f"Scatter Plot: {cols[0]} vs {cols[1]}"
                    )
                else:
                    st.warning("Select at least 2 columns for Scatter plot")

            # If a valid figure was created, display it using Streamlit
            if fig:
                st.plotly_chart(fig, use_container_width=True)
                # use_container_width=True makes the chart responsive to app layout

st.subheader("Upload CSV to IRIS")

# File uploader for CSV files
uploaded_file = st.file_uploader("Choose a CSV file", type="csv")

if uploaded_file:
    # Read and preview the uploaded CSV
    csv_df = pd.read_csv(uploaded_file)
    st.dataframe(csv_df.head())

    # Input: full table name in format "Namespace.Table"
    full_table_name = st.text_input("Enter target IRIS table (e.g. Bank.Account)")

    if st.button("Insert Data"):
        try:
            # Split schema (namespace) and table name if dot notation is used
            if '.' in full_table_name:
                schema, table_name = full_table_name.split('.', 1)
            else:
                schema = None
                table_name = full_table_name

            # Attempt to insert data into existing table
            with engine.begin() as conn:
                csv_df.to_sql(
                    name=table_name,
                    con=conn,
                    if_exists='append',
                    index=False,
                    schema=schema,
                    method='multi'  # batch insert for IRIS
```

```
            )

        st.success(f"Successfully inserted data into '{full_table_name}'")

    except Exception as e:
        st.error(f"Insertion failed: {e}")
```

7

Build a RESTful Application with Flask

In the previous exercise, you learned how to build an interactive application using Streamlit. This section introduces how to use Flask to expose InterSystems IRIS data via RESTful APIs.

7.1 Building RESTful Flask Applications with InterSystems IRIS

Flask is a lightweight and flexible Python web framework, ideal for developing REST APIs and back-end services. When paired with SQLAlchemy and InterSystems IRIS, Flask makes it easy to expose InterSystems IRIS data to front-end applications, services, and other consumers. This guide focuses on the InterSystems IRIS-specific setup steps so that you can get started quickly, and then proceed as with any other Flask app.

7.2 Prerequisites

Before starting this exercise, make sure that you have installed SQLAlchemy and DB-API using the following command:

```
pip install sqlalchemy-intersystems-iris
```

See the DB-API exercise for a brief description of these tools.

You also need to install Flask:

```
pip install flask
```

You should also have access to a running InterSystems IRIS instance and valid connection credentials (host, port, username, password, namespace).

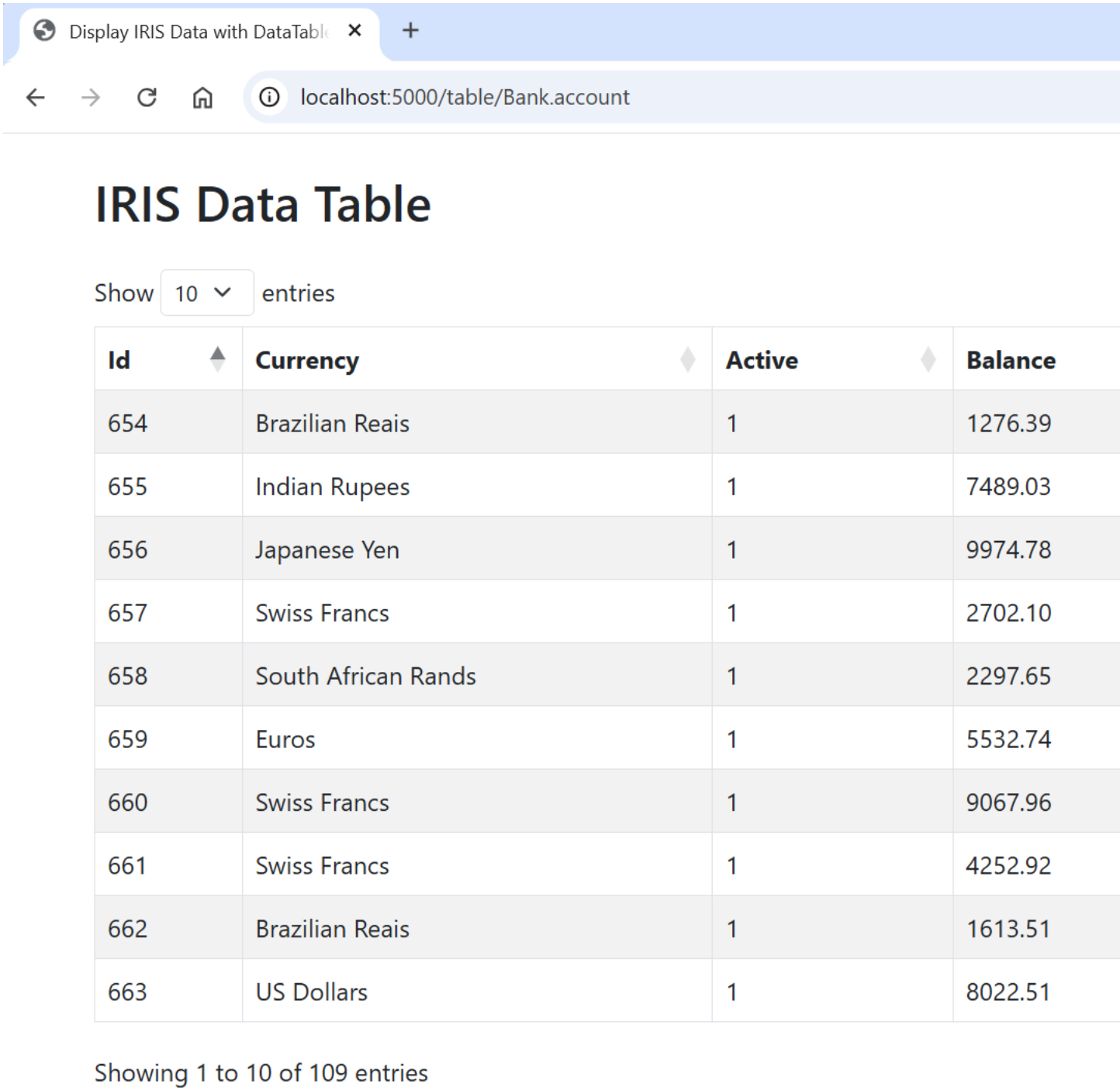
7.3 What You Learn

- How to set up a basic Flask application.
- How to connect Flask to InterSystems IRIS using SQLAlchemy.

- How to use HTML templates and render data from InterSystems IRIS.

7.4 What You Build

By the end of this documentation, you have a Flask web application that allows you to view your data in InterSystems IRIS through a clean user interface in the web browser.



7.5 Creating Your First Flask App

To get started with Flask, you create a minimal web application that returns a simple message in the browser.

First, create a new directory for your project (`my_flask`). Inside it, create a file named `app.py` with the following content:

Python

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello World!</p>"

if __name__ == '__main__':
    app.run(debug=True)
```

In the code above:

- `Flask(__name__)` creates the Flask application instance.
- `@app.route("/")` decorator defines the route for the root URL (`/`).
- `hello_world()` is the view function returning a simple HTML message.
- `app.run(debug=True)` starts the local development server with debugging enabled.

In Flask, routes map URLs to their respective functions (as defined by `@app.route("<URL>")`). Accessing these URLs triggers their associated functions. The `/` URL calls the home function (typically at `http://127:0.0.1:5000/`). To call other functions via other URLs (defined by `@app.route("<URL>")`), open `http://127:0.0.1:5000/<URL>` on your browser.

Important: Do not name your application as `flask.py` for doing so creates a conflict with Flask itself.

7.6 Running the Flask Application

From the directory containing `app.py`, run the following in the command line:

```
python app.py
```

This starts the server, and you can now access your app at `http://127:0.0.1:5000/`.

7.7 Using HTML Templates

Web applications typically use HTML for rendering pages. Flask integrates the Jinja2 templating engine to separate your Python logic from HTML code. This separation improves code readability, maintainability, and reusability.

Understanding the detailed syntax and structure of HTML, CSS, and JavaScript is out of scope for this guide. Just know that you can use them within your Flask app to build dynamic, styled, and interactive web pages.

To implement an HTML template, follow these steps:

1. Create a directory named `templates` the same directory as `app.py`.

2. Inside `templates`, create a file named `index.html` with this content:

```
<!DOCTYPE html>
<html>

<body>
    <h1>My First Heading</h1>
    <p>My First Paragraph</p>
</body>

</html>
```

3. Modify `app.py` to render the template.

Python

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def hello_world():
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)
```

The following happens in the code above:

- When a user accesses the root URL (`/`), Flask triggers the `hello_world()` function.
- This function calls `render_template('index.html')`, which tells Flask to load and return the HTML from the `templates/`directory.

The `templates/` directory is Flask's default location for all HTML templates. Flask separates the front end (HTML/CSS/JS) from the back end (Python), making your codebase cleaner and easier to manage. This modular approach also makes it easier to scale your application as you add more pages and templates.

7.8 Directory Structure

This is a typical minimal Flask application layout. Each file and directory serve a specific role to separate the different parts of the application code and keep the project organized and maintainable.

```
my_flask/
│
├── app.py          # Main application logic
├── models.py       # SQLAlchemy models (optional but recommended)
├── templates/      # HTML templates
│   └── index.html  # Main HTML page rendered by Flask
├── static/         # CSS, JS, images (optional)
```

7.8.1 What Each Directory Component Is

- `app.py`

  This is the entry point of the Flask application. It contains the route definitions, application configuration, and logic to start the server. It typically handles request routing and renders templates.

- `models.py`

  Contains the SQLAlchemy model definitions, which map Python classes to database tables. This helps abstract and manage database interactions cleanly.

  Note: While `models.py` is not strictly required, it helps organize your Object Relational Mapping logic in a clean and modular way.

- `templates/`

  Flask uses Jinja2 templating and all HTML files go in this directory. The framework automatically looks for templates here when rendering views using `render_template()`.

- `index.html`

  A specific HTML file inside the templates directory, typically used as the home page or main data table view. This is where your front end DataTable integration (like in `render_template()`) would live.

- `static/`

  Optional but useful for storing static files like custom CSS, JavaScript, images, or fonts. Flask serves these files from the `static/URL` path automatically.

Flask intentionally does not enforce strict structures, so while this layout is clean and scalable, you are free to adapt it based on your application's needs.

7.9 Connecting to InterSystems IRIS

To connect to InterSystems IRIS using SQLAlchemy, define a connection string and create an engine. This engine object is reused throughout the app to run queries and insert data and acts like a persistent, reusable pipeline to your database.

Python

```
from sqlalchemy import create_engine, text

# Replace with your credentials and connection information
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

engine = create_engine(DATABASE_URL, echo=True) # Set echo=True to see the SQL queries being executed
 in the terminal
```

For more documentation on using SQLAlchemy with InterSystems IRIS, see InterSystems IRIS and SQLAlchemy.

7.10 Transferring Data from InterSystems IRIS to Flask and Displaying it

7.10.1 tables.html

The following provides the web page structure for viewing the InterSystems IRIS data. It utilizes jQuery DataTables, a popular JavaScript library used to create dynamic, interactive tables with features like pagination, sorting, searching, and

responsive layouts. It also incorporates Bootstrap, a modern CSS framework that provides a responsive grid system, pre-styled UI components, and utility classes. Bootstrap is used here to style the table and layout elements (such as spacing and table orders), ensuring the table looks clean and is mobile-responsive.

```
<!DOCTYPE html>
<html>

<head>
    <title>Display InterSystems IRIS Data with DataTables</title>

    <!-- Bootstrap 5 for styling -->
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">


    <!-- jQuery (required by DataTables) -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

    <!-- DataTables + Bootstrap 5 integration -->
   <link rel="stylesheet" href="https://cdn.datatables.net/1.13.5/css/dataTables.bootstrap5.min.css">

    <script src="https://cdn.datatables.net/1.13.5/js/jquery.dataTables.min.js"></script>
    <script src="https://cdn.datatables.net/1.13.5/js/dataTables.bootstrap5.min.js"></script>
</head>

<body class="container py-4">

    <h2 class="mb-4">InterSystems IRIS Data Table</h2>

    <!-- HTML table to be populated dynamically -->
    <table id="myTable" class="table table-bordered table-striped table-hover"></table>

    <script>
        $(document).ready(function () {
            // Convert server-passed JSON strings into JavaScript objects
            let my_data = JSON.parse('{{ my_data | tojson | safe }}');
            let my_cols = JSON.parse('{{ my_cols | tojson | safe }}');

            // Initialize DataTable with server data and column definitions
            $('#myTable').DataTable({
                data: my_data,
                columns: my_cols,
                responsive: true,
                lengthMenu: [5, 10, 25, 50],
                pageLength: 10,
                language: {
                    search: "_INPUT_",
                    searchPlaceholder: "Search records"
                }
            });
        });
    </script>

</body>

</html>
```

Just like with `index.html`, understanding the code above is not within the scope of this documentation. The takeaway here is that you can integrate these tools into your Flask application. Refer to the official documentation of these libraries to learn more about them.

7.10.2 app.py

Create a dynamic view into your InterSystems IRIS data through the `/table/` URL route. Using a connection via the SQLAlchemy engine to your InterSystems IRIS database, query data from the specified table and populate it into your Flask app. Pass the resulting data to the HTML template from above to create a clean and structured display of it in the web page.

You can specify which table to query directly through the URL. For example, to view data from a table called `Bank.Account`, open `http://127:0.0.1:5000/table/Bank.Account` on your browser.

The app :

1. Parses the table name (optionally including the schema).

2. Executes a `SELECT * FROM table ORDER BY 1` query.
3. Converts the results into a list of dictionaries.
4. Dynamically extracts column names.
5. Renders the `tables.html` template with your data and columns.

Python

```
@app.route("/table/<path:table_name>")
def show_table(table_name):
    # table_name could be "Schema.Table" or just "Table"
    try:
        # Validate and split schema/table if schema is provided
        if '.' in table_name:
            schema, table = table_name.split('.')
            full_table = f"{schema}.{table}"
        else:
            schema = None
            table = table_name
            full_table = table

        # Query to get data from the specified table
        query = f"SELECT * FROM {full_table} ORDER BY 1"

        with engine.connect() as conn:
            result = conn.execute(text(query))
            keys = result.keys()  # get column names
            rows = [dict(zip(keys, row)) for row in result]

        if not rows:
            return f"No data found in table {full_table}", 404

        # Extract columns dynamically from first row keys
        columns = [{"data": col, "title": col.capitalize()} for col in rows[0].keys()]

        return render_template("tables.html", my_data=rows, my_cols=columns)

    except Exception as e:
        return f"Error: {str(e)}", 500
```

In the code above:

- `<path:table_name>` defines a Flask route where `table_name` can include dots (for example, `Bank.Account`), The `:path` converter allows such values to be passed as arguments into the view function.
- Flask uses the Jinja2 templating engine, which allows you to pass arguments (like `rows` and `columns`) from your view function into your HTML templates. These variables can be inserted dynamically using the syntax `{{variable}}` (see `tables.html` as an example).
- `if '.' in table_name` splits the table into `schema` and `table` components when the format includes a dot.
- `rows = [dict(zip(keys,row) for row in result]` passes both the row data and column definitions into the HTML template, enabling a dynamic and responsive table view.

  Note: The `result` returned by `conn.execute()` is not a dictionary. Instead, it is an iterable of row tuples or RowProxy objects. To make data easier to work with in Jinja2 templates, the code converts each row to a dictionary by pairing column names with values using `zip()`.

This application enables you to quickly browse and visualize any table in your InterSystems IRIS database by simply modifying the URL. It dynamically pulls and formats tables using SQLAlchemy, then renders it in a styled HTML table using Jinja2. The use of dynamic route arguments and template variables makes it flexible for inspecting a wide range of tables without modifying the back-end logic.

This documentation only touches the surface of Flask. Now that you know how to get started creating a Flask application with InterSystems IRIS, you can fully utilize all of Flask's features just as with any other Flask application.

To dive deeper into Flask, visit the official Flask documentation.

7.11 Complete Flask Code

The following is the code for the Flask application built. Be aware of the file structure required to have the application running smoothly.

Python

```
#app.py

from flask import Flask, render_template
from sqlalchemy import create_engine, text

# Replace with your credentials and connection information
username = "_SYSTEM"
password = "SYS"
namespace = "USER"
DATABASE_URL = f"iris://{username}:{password}@localhost:1972/{namespace}"

engine = create_engine(DATABASE_URL, echo=True) # Set echo=True to see the SQL queries being executed
 in the terminal

app = Flask(__name__)

# ---------- ROUTES ----------
@app.route("/")
def hello_world():
    # main index page
    return render_template("index.html")

@app.route("/table/<path:table_name>")
def show_table(table_name):
    # table_name could be "Schema.Table" or just "Table"
    try:
        # Validate and split schema/table if schema is provided
        if '.' in table_name:
            schema, table = table_name.split('.')
            full_table = f"{schema}.{table}"
        else:
            schema = None
            table = table_name
            full_table = table

        # Query to get data from the specified table
        query = f"SELECT * FROM {full_table} ORDER BY 1"

        with engine.connect() as conn:
            result = conn.execute(text(query))
            keys = result.keys()  # get column names
            rows = [dict(zip(keys, row)) for row in result]

        if not rows:
            return f"No data found in table {full_table}", 404

        # Extract columns dynamically from first row keys
        columns = [{"data": col, "title": col.capitalize()} for col in rows[0].keys()]

        return render_template("tables.html", my_data=rows, my_cols=columns)

    except Exception as e:
        return f"Error: {str(e)}", 500


<!-- index.html -->

<!DOCTYPE html>
<html>

<body>
    <h1>My First Heading</h1>
    <p>My First Paragraph</p>
</body>

</html>

<!-- tables.html -->

<!DOCTYPE html>
<html>
```

```
<head>
    <title>Display IRIS Data with DataTables</title>

    <!-- Bootstrap 5 for styling -->
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">


    <!-- jQuery (required by DataTables) -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

    <!-- DataTables + Bootstrap 5 integration -->
   <link rel="stylesheet" href="https://cdn.datatables.net/1.13.5/css/dataTables.bootstrap5.min.css">

    <script src="https://cdn.datatables.net/1.13.5/js/jquery.dataTables.min.js"></script>
    <script src="https://cdn.datatables.net/1.13.5/js/dataTables.bootstrap5.min.js"></script>
</head>

<body class="container py-4">

    <h2 class="mb-4">IRIS Data Table</h2>

    <!-- HTML table to be populated dynamically -->
    <table id="myTable" class="table table-bordered table-striped table-hover"></table>

    <script>
        $(document).ready(function () {
            // Convert server-passed JSON strings into JavaScript objects
            let my_data = JSON.parse('{{ my_data | tojson | safe }}');
            let my_cols = JSON.parse('{{ my_cols | tojson | safe }}');

            // Initialize DataTable with server data and column definitions
            $('#myTable').DataTable({
                data: my_data,
                columns: my_cols,
                responsive: true,
                lengthMenu: [5, 10, 25, 50],
                pageLength: 10,
                language: {
                    search: "_INPUT_",
                    searchPlaceholder: "Search records"
                }
            });
        });
    </script>

</body>

</html>
```

8

Code Interactively with Jupyter Notebooks

In addition to Streamlit and Flask, you can also use Jupyter Notebooks with InterSystems IRIS.

8.1 Interactive Exploration with Jupyter Notebooks

Jupyter Notebooks are interactive coding environments often used for data science, machine learning, and teaching. They combine code, visualization, and markdown-based documentation in a single browser-based interface. With your notebooks, you can connect to InterSystems IRIS and analyze data using standard Python libraries such as pandas, matplotlib, and more. Using Jupyter Notebooks with InterSystems IRIS is no different than using any other database. Watch Using Embedded Python as a Jupyter Notebook Server to see an example of this integration.

9

Become an InterSystems IRIS Power User

Once you are comfortable working with Python and InterSystems IRIS® data platform, you are ready to take the next step and unlock the full potential of the platform. InterSystems IRIS is far more than a database. It is a unified data platform that offers powerful, integrated capabilities across interoperability, analytics, multi-model storage, and AI-enablement. InterSystems IRIS delivers the scale and speed of modern cloud-native systems, the interoperability of enterprise middleware, and the performance of an in-process analytics platform, all in one cohesive environment.

This section introduces you to these advanced features which allow you to build smarter, faster, and more connected applications. Becoming an InterSystems IRIS power user means expanding beyond traditional development patterns and leveraging the architectural strengths of this platform.

9.1 Building Interoperability Productions with Python

InterSystems IRIS is a platform designed for integration and orchestration. Its interoperability features—such as message routing, persistent queues, and businesses processes—can all be extended using Python.

9.1.1 Production EXtension Framework (PEX)

Using the Python Gateway and PEX framework, you can embed Python code directly into your interoperability productions. This enables real-time interaction with Python-based services. For example:

- You can create a business service in Python that ingests data from an external API.
- You can define a business process that calls a Python machine learning model to make a decision.
- You can develop a business operation in Python that pushes transformed data to an external system.

If you are building integrations between systems, Python can now be a full participant in your InterSystems IRIS production pipelines, meaning that Python code can participate in messaging, tracing, error handling, and orchestration like any native component. The gateway-based approach allows InterSystems IRIS to invoke external Python modules as part of real-time workflows, making it ideal for hybrid architecture or multi-system data flows.

9.1.2 InterSystems Python Productions (PyProd)

InterSystems Python Productions is a Python library that enables you to build interoperability components entirely in Python. It is available from PyPi: intersystems_pyprod.

With PyProd, you can write your production components in a regular Python script, importing the required base classes from intersystems_pyprod and defining your own components by subclassing them, just as you would with any other Python

library. Then, load your Python classes from the command line to link them with InterSystems IRIS so that they appear as production components that can be configured from the Management Portal.

Whether you are developing entirely new “Pure Python” productions or enhancing existing productions, PyProd ensures that your components remain fully integrated, observable, and easy to configure.

Note: InterSystems PyProd is an experimental feature.

9.2 Taking Advantage of InterSystems IRIS’s Multi-Model and Analytics Features

InterSystems IRIS supports multi-model data access, allowing you to store and query data using relational, object, document, and key-value paradigms all in one system. Its architecture supports real-time analytics, embedded business intelligence, vector search, and even natural language processing—all from a single platform. You combine these models to suit your application’s needs and access them from Python.

9.2.1 Some Key Capabilities to Explore as a Power User

- *Integrated Business Intelligence*: InterSystems IRIS includes dashboards and pivot tables that you can integrate into your applications.
- *Vector Search*: InterSystems IRIS supports native vector storage and indexing for AI workloads, such as retrieval-augmented generation (RAG). You can build hybrid applications combining Python LLM orchestration with in-IRIS vector search for high performance using libraries such as LangChain and Hugging Face.
- *Adaptive Analytics*: InterSystems IRIS provides a virtual data model layer between InterSystems IRIS, Business Intelligence, and Artificial Intelligence client tools. This common data model abstracts away differing definitions and calculations, providing a unified approach to working with your business operations.

9.2.2 Mastering Globals: Unlocking High-Performance Data Modeling

To truly harness InterSystems IRIS’s performance capabilities, it is worth understanding globals, the native multi-dimensional, hierarchical data structures at the core of InterSystems IRIS.

Globals offer:

- Schema-less, key-value styled storage with deep nesting.
- Extremely fast read-write performance even at large scales.
- Support for real-time telemetry, hierarchical document modeling, and non-relational use cases.

While not required for typical SQL-based applications, globals are key reason why many high-throughput systems run on InterSystems IRIS. Python can interact with globals via the Native API (out-of-process) or Embedded Python (in-process), making it easier to explore and prototype new storage models.

If you are building performance-critical applications, globals are an indispensable part of the InterSystems IRIS toolkit. Understanding how globals work will elevate your ability to optimize performance, customize data structures, and take full advantage of the InterSystems IRIS engine.

9.2.3 ObjectScript: The Native Language of InterSystems IRIS

ObjectScript is the native language of InterSystems IRIS. Just like Python, ObjectScript is an object oriented dynamic, interpreted language with full polymorphic dispatch. Written in C, both languages use reference counting for object lifetimes. These similarities make it easy to work with the two languages together. However, once you begin exploring ObjectScript, you will find a language that is expressive, compact, and designed specifically for data-centric programming. Though you can utilize 90% of the distinguishing features of InterSystems IRIS by using Embedded Python, consider using ObjectScript to completely make use of InterSystems IRIS.

9.2.3.1 What Makes ObjectScript Special

- It is optimized for data operations: Globals, objects, indexes, and storage strategies are all native to the language.
- You can write compact, high-performance logic with less code than with other languages.
- It allows seamless translation between different data storage paradigms.

9.2.3.2 Embracing the Best of All Worlds: Python, ObjectScript, and Globals

Being a power user in InterSystems IRIS does not mean choosing one language or access method over another. It means knowing when and how to use each. With Embedded Python, you can combine the best of modern scripting with the unique strengths of InterSystems IRIS.

Here are some practices to consider:

- Use SQL for declarative access and compatibility.
- Use Globals for unmatched performance and flexibility.
- Use Python for AI, modeling, data science, or custom algorithms.
- Use ObjectScript to bridge it all, especially when building classes, services, and complex business logic inside the InterSystems IRIS process.

InterSystems IRIS is unique because it does not force you to choose between modernity and performance, or openness and reliability. It invites you to build complete systems, with the freedom to code where you are most comfortable, and the power to go deeper when you need to.

9.3 Moving forward

By stepping into these capabilities, you are no longer just writing Python code that connects to a database. Rather, you are building intelligent, integrated systems that can scale, adapt, and evolve. InterSystems IRIS is different by design, and by learning to use its full platform capabilities through Python, you become a different kind of developer: a power user capable of building data-driven applications that move faster and do more.

Your Python journey with InterSystems IRIS does not end here. There are always new and exciting Python InterSystems IRIS applications being created, whether they are official productions developed by InterSystems or open-source projects created by members on the OpenExchange. Join the InterSystems Developer Community to immerse yourself in the growing Python developer community and continue learning more by visiting the Online Learning platform and checking out more in documentation.